D03｜回复丢了，怎样仍只做一遍

对方创建后丢了回复，本地只知道没拿到可用结果。**同一意图多次传输，外部效果仍一次**，要靠持久化的业务身份、不可变输入，以及对方真正实现的去重合同共同成立。

重传时，到底哪些东西必须不变？

当前问题	关系或短用法	记到什么程度
第一次超时，外部案件可能已存在吗？	对方先提交 P-0042，再丢回复，本地仍 pending；原 key 重传取回 P-0042，得到 2 次调用、1 次效果，本地随后确认。	**timeout 不会擦掉远端提交。** 本地状态与对方事实分别读。
为什么第一次发送前就提交完整 intent？	保存意图 ID、案件 ID、合同版本、冻结 canonical JSON、摘要、稳定 key 与 pending，进程退出后才有材料恢复同一动作。	要能说明每项为何一起存在；字段全名与建表配方查用。
只保存摘要或 key，够重放吗？	摘要不能还原原输入；只存 key 却从已修改表单重建内容，也可能把新业务塞给旧身份。	**重放读取冻结输入；不随 attempt 重建或覆盖摘要。**
request ID 换了，要换幂等键吗？	request ID 标识一次处理；同 intent 的后续处理可用新 request ID，仍沿用原 key、合同和内容。	分清处理身份与业务身份的寿命，不把日志关联当去重。
用户改了真实业务内容怎么办？	新内容用新 intent、新冻结输入、新摘要、新 key；旧记录保留。新 key 可以产生新效果，但不会取消旧案件。	旧意图不能被改写；是否取消旧效果属于另一项明确动作。

规范化把当前合同内相同内容写成确定 JSON 表示，如键排序、去掉分隔空格；再对 UTF-8 字节做 SHA-256。摘要用来比较，不是加密，也不是原文备份。日期、浮点数、默认值等规则变化时，必须核对合同版本，不能把这份小配方当所有系统通用标准。

回看：「先看两边各自留下了什么」「为什么第一次调用以前就要保存完整意图？」「新的一次请求，不一定是新的一件事」。

找到旧 pending，是否已经有资格发送？

当前问题	关系或短用法	记到什么程度
有当前版本、完整输入且摘要相同，就自动重发吗？	这些只提供重放材料；还要符合 provider 的 key 有效期、查询方式、并发限制和有限策略。本例由新的本地 POST 显式触发恢复，GET 不发送。	**可恢复不等于一见 pending 就自动执行。** 材料不足时保持未知并交核对。
同 key 不同内容，谁来拒绝？	本地先挡住意图冲突；对方也要持久保存 key、原输入和结果，检测冲突并拒绝。任意 POST 加一个 header 不会自动获得保护。	两端责任都要有证据；具体 header、保留期限、冲突格式接入时查。
本地事务能把远端一起回滚吗？	先用短事务保存 pending，再调用，再用短事务确认；本地与 provider 各自提交，任何一方 rollback 都不自动撤回另一方。	**网络等待不把两库绑成一个事务。** 连接管理细节可查。
远端成功，本地 confirm 写失败，怎么接回？	保留此前完整 pending，暴露本地存储故障；后续按原输入与 key 取回原结果，再尝试本地确认。	不生成新 key 补做；连人工记录也未保存时不能声称“已转人工”。

完整示例以两份临时 SQLite 保存两边事实。重新连接读回可验证已提交，但顺序恢复实验不覆盖突然断电、key 过期、多进程竞争或所有并发。第一次恢复的 `2 / 1 / partner_confirmed` 与随后故意制造冲突、新意图的累计次数分开看。

回看：「为什么“有一个 key”还不够？」「如果远端成功了，本地写入却失败呢？」「两份临时数据库怎样把这个过程留下来？」。